Embedded system building tools



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Approaches

Three main approaches to build your embedded Linux system:

1. Cross-compile everything manually from source
2. Use a binary distribution such as Debian, Ubuntu or Fedora
3. Use an *embedded Linux build system* that automates the cross-compilation process



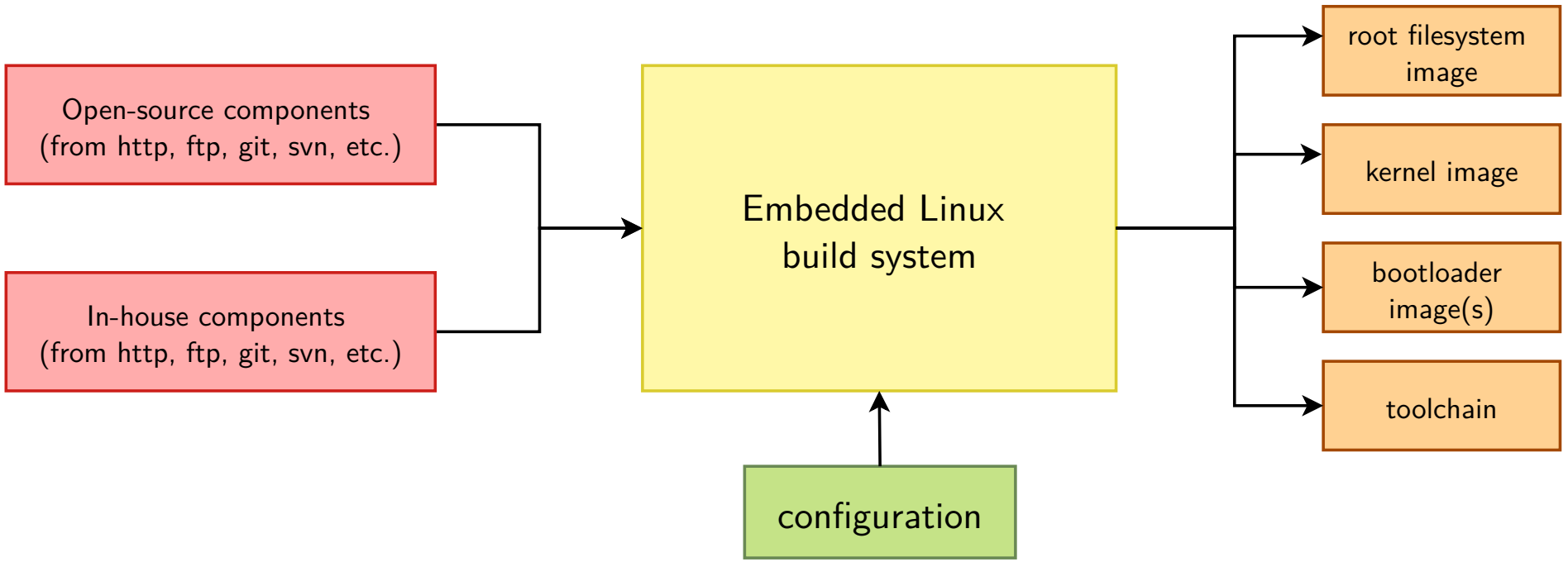
Approaches pros and cons

	Pros	**Cons**
Building everything manually	Full flexibility Learning experience	Dependency hell Need to understand a lot of details Version compatibility Lack of reproducibility
Binary distribution Debian, Ubuntu, Fedora, etc.	Easy to create and extend Extensive set of packages Usually excellent security maintenance	Hard to customize Hard to optimize (boot time, size) Hard to rebuild the full system from source Large system Uses native compilation (slow) No well-defined mechanism to generate an image Lots of mandatory dependencies Not available for all architectures
Embedded Linux Build systems Buildroot, Yocto, PTXdist, OpenWrt, etc.	Nearly full flexibility Built from source: customization and optimization are easy Fully reproducible Uses cross-compilation Have embedded specific packages not necessarily in desktop distros Make more features optional	Not as easy as a binary distribution Build time



Embedded Linux build systems

Embedded Linux build system: principle

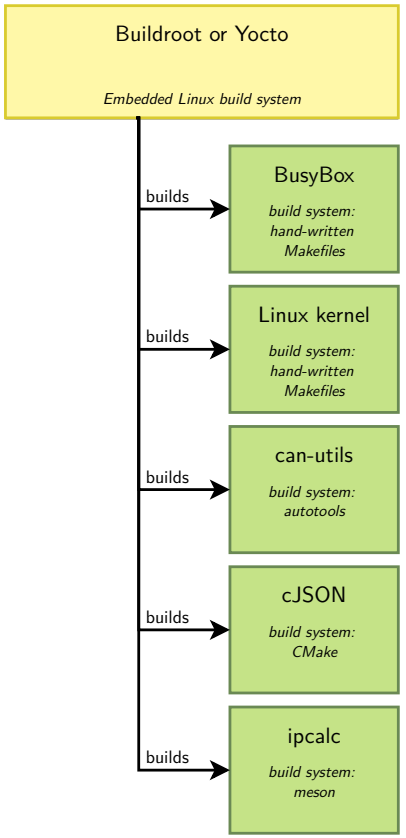


- Building from source → lot of flexibility
- Cross-compilation → leveraging fast build machines
- Recipes for building components → easy



Build systems vs. Embedded Linux build systems

- Possible confusion between *build system* (Makefiles, autotools, CMake, Meson) and *embedded Linux build systems* (Buildroot, Yocto/OpenEmbedded, OpenWrt, etc.)
- *Build systems* are used by individual software components, to control the build process of each source file into a library, executable, documentation, etc.
- *Embedded Linux build systems* are tools that orchestrate the build of all software components one after the other. They invoke the *build system* of each software component.





Buildroot: introduction

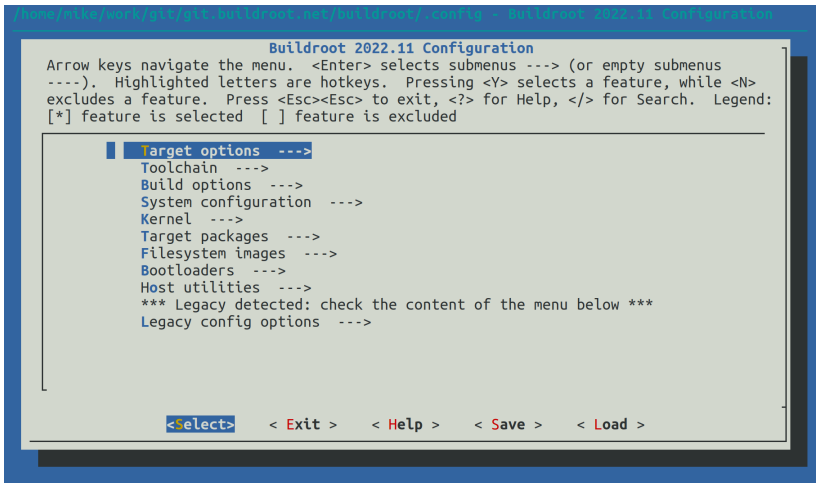


- Allows to build a toolchain, a root filesystem image with many applications and libraries, a bootloader and a kernel image
  - Or any combination of the previous items
- Supports using uClibc, glibc and musl toolchains, either built by Buildroot, or external
- Over 2800 applications or libraries integrated, from basic utilities to more elaborate software stacks: Wayland, GStreamer, Qt, Gtk, WebKit, Python, PHP, NodeJS, Go, Rust, etc.
- Good for small to medium size embedded systems, with a fixed set of features
  - No support for generating packages (`.deb` or `.ipk`)
  - Needs complete rebuild for most configuration changes.
- Active community, releases published every 3 months. One LTS release made every 2 years, maintained for 3 years.



Buildroot: configuration and build

- Configuration takes place through a `*config` interface similar to the kernel `make menuconfig`
- Allows to define
  - Architecture and specific CPU
  - Toolchain configuration
  - Set of applications and libraries to integrate
  - Filesystem images to generate
  - Kernel and bootloader configuration
- Build:
  `make`
- Useful build results in `output/images/`





Buildroot: adding a new package

- A package allows to integrate a user application or library to Buildroot
- Can be used to integrate
  - Additional open-source libraries or applications
  - But also your own proprietary libraries and applications → fully integrated build process
- Each package has its own directory (such as `package/jose`). This directory contains:
  - A `Config.in` file (mandatory), describing the configuration options for the package. At least one is needed to enable the package. This file must be sourced from `package/Config.in`
  - A `jose.mk` file (mandatory), describing how the package is built.
  - A `jose.hash` file (optional, but recommended), containing hashes for the files to download, and for the license file.
  - Patches (optional). Each file of the form `*.patch` will be applied as a patch.



Buildroot: adding a new package, Config.in

package/jose/Config.in

```
config BR2_PACKAGE_JOSE
        bool "jose"
        depends on BR2_TOOLCHAIN_HAS_THREADS
        select BR2_PACKAGE_ZLIB
        select BR2_PACKAGE_JANSSON
        select BR2_PACKAGE_OPENSSL
        help
          C-language implementation of Javascript Object Signing and
          Encryption.

          https://github.com/latchset/jose
```

package/Config.in

```
[...]
source "package/jose/Config.in"
[...]
```



Buildroot: adding new package, `.mk` file

package/jose/jose.mk

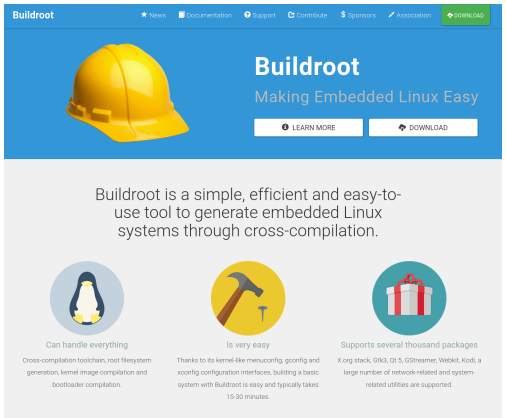
```
JOSE_VERSION = 11
JOSE_SOURCE = jose-$(JOSE_VERSION).tar.xz
JOSE_SITE = https://github.com/latchset/jose/releases/download/v$(JOSE_VERSION)
JOSE_LICENSE = Apache-2.0
JOSE_LICENSE_FILES = COPYING
JOSE_INSTALL_STAGING = YES
JOSE_DEPENDENCIES = host-pkgconf zlib jansson openssl

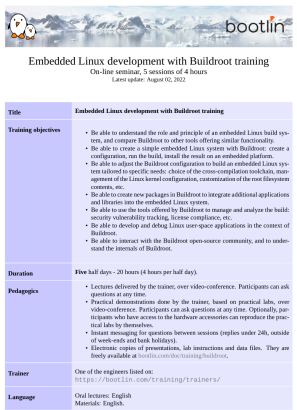
$(eval $(meson-package))
```

- The package directory and the prefix of all variables must be identical to the suffix of the main configuration option `BR2_PACKAGE_JOSE`
- The `meson-package` infrastructure knows how to build *Meson* packages. Many other infrastructures exist, for different *build systems*

Buildroot resources

- Official site: https://buildroot.org/
- Buildroot manual: https://buildroot.org/downloads/manual/manual.html
- Complete *Buildroot system development* training course from Bootlin
  - https://bootlin.com/training/buildroot/
  - Freely available training materials

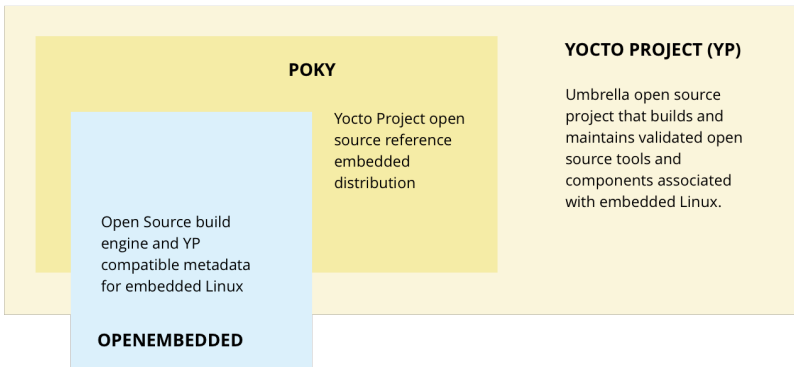






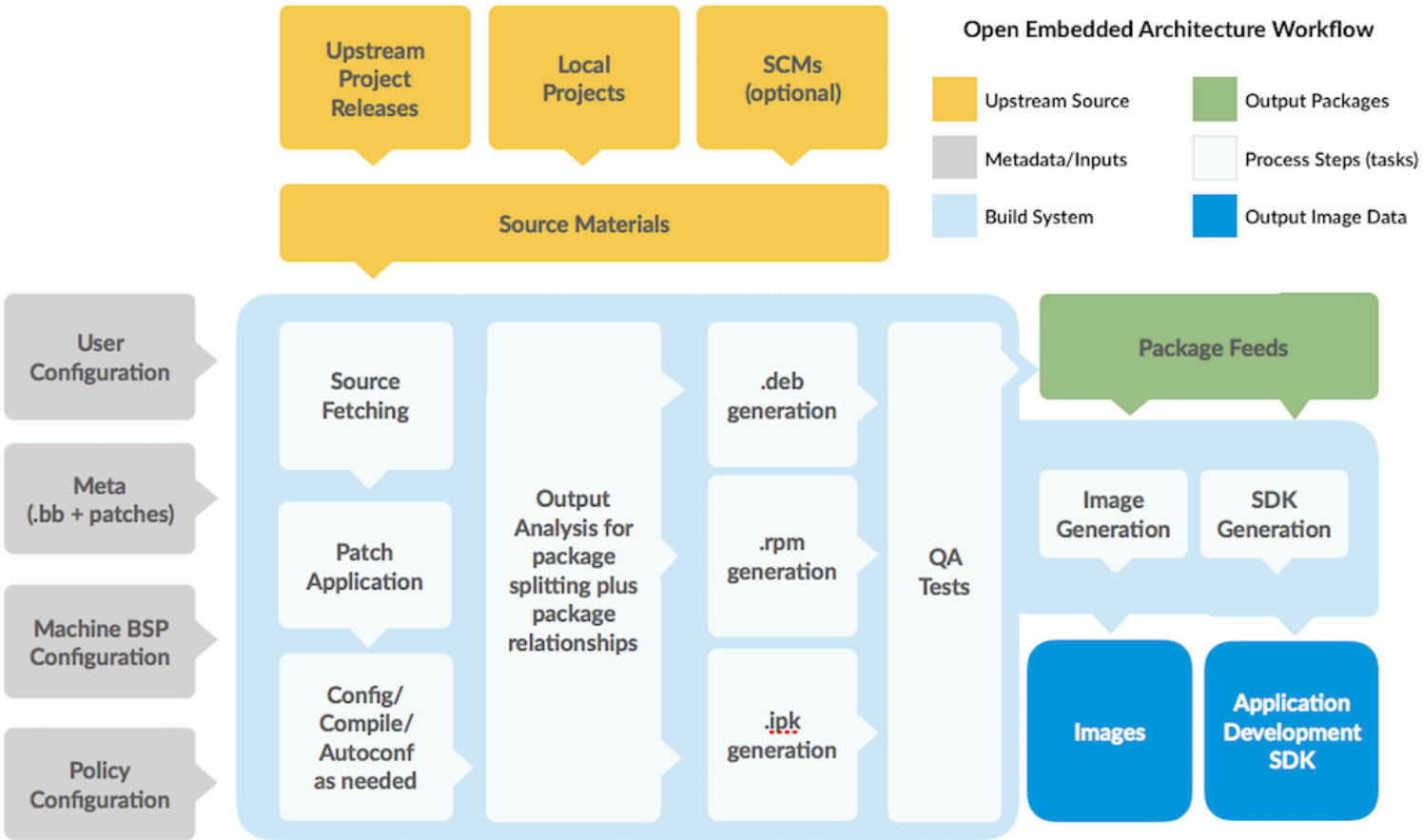
Yocto Project / OpenEmbedded

- OpenEmbedded
  - Started in 2003
  - Goal is to build custom Linux distributions for embedded devices
  - Back then, no stable releases, limited/no documentation, difficult to use for products
- Yocto Project
  - Started in 2011
  - By the Linux Foundation
  - Goal is to *industrialize* OpenEmbedded
  - Funds the development of OpenEmbedded, makes regular stable releases, QA effort, extensive documentation
  - One Long Term Support release every 2 years, supported for 4 years.





Yocto Project overview





Yocto Project concepts

- Terminology
  - **Layer**: Git repository containing a collection of recipes, machines, etc.
  - **Recipe**: metadata that describes how to build a particular software component, the contents of an image to generate
  - **Machine**: a specific hardware platform
  - **bitbake**: the orchestration tool that processes *recipes* to generate the final products
- Yocto/OpenEmbedded generate a *distribution*
  - For each recipe, it produces one or several binary packages (`deb`, `rpm`, `ipk`)
  - A selection of these binary packages are installed to create a *root filesystem image* that can be flashed
  - The other packages can be installed at runtime on the system using a package management system: `apt`, `dnf`, `opkg`



Public layers (1/2)

- Core layers
  - bitbake, not really a layer, but the core build orchestration tool
  - openembedded-core, the very core recipes, to build the most common software packages: Linux, BusyBox, toolchain, systemd, mesa3d, X.org, Wayland bootloaders. Supports only QEMU machines.
  - poky, a layer from the Yocto Project that defines the *Poky* distribution, a reference distribution. In practice not useful for real projects.
  - meta-openembedded, community maintained additional recipes from the OpenEmbedded project
- BSP layers, provided by HW vendors or the community, to support additional hardware platforms: recipes for building custom Linux kernel, bootloaders, for HW-related software components
  - meta-intel, meta-arm, meta-ti, meta-xilinx, meta-freescale, meta-atmel, meta-st-stm32mp, etc.



Public layers (2/2)

- Additional software layers: recipes for building additional software components, not in *openembedded-core*
  - meta-qt6, meta-virtualization, meta-rauc, meta-swupdate, etc.
- Layer index: https://layers.openembedded.org/
- Each layer normally has a branch matching the Yocto release you're using
- Not all layers have the same level of quality/maintenance: third-party layers are not necessarily reviewed by OpenEmbedded experts.



Combine layers

- For your project, you will typically combine a number of public layers
  - At least the *openembedded-core* layer
  - Possibly one or several *BSP layers*
  - Possibly one or several additional *software layers*
- And you will create your *own layer*, containing recipes for:
  - Machine definitions for your custom hardware platforms
  - Image/distro definitions for your custom system(s)
  - Recipes for your custom software
- A tool is often used to automate the retrieval of the necessary layers, at the right version
  - Google repo tool, the Yocto-specific Kas utility



Yocto quick start: STM32MP1 example

Download *bitbake* and layers

```
$ git clone https://git.openembedded.org/openembedded-core
$ git -C openembedded-core checkout e67d659847af
$ git clone https://git.openembedded.org/meta-openembedded
$ git -C meta-openembedded checkout 4052c97dc83d
$ git clone https://git.openembedded.org/bitbake -b 2.0
$ git clone https://github.com/STMicroelectronics/meta-st-stm32mp.git
    -b openstlinux-5.15-yocto-kirkstone-mp1-v23.07.26
```

Note: we're not using a tool such as *repo* or *Kas* here, we are fetching each layer manually. Enter the build environment

```
$ source openembedded-core/oe-init-build-env
```

This automatically enters a directory called `build/`, with a few files/directories already prepared.



Yocto quick start: STM32MP1 example

Configure layers: *conf/bblayers.conf*

```
BBLAYERS ?= "
  /path/to/openembedded-core/meta
  /path/to/meta-st-stm32mp
  /path/to/meta-openembedded/meta-oe
  /path/to/meta-openembedded/meta-python
  "
```

Start the build

```
$ MACHINE=stm32mp1 bitbake core-image-minimal
```

- `MACHINE=stm32mp1` will build images usable on all STM32MP1 platforms
- `core-image-minimal` builds a minimal image

Build results

```
$ ls tmp-glibc/deploy/images/stm32mp1/
```



Yocto recipe example

`openembedded-core/tree/meta/recipes-extended/libmnl/libmnl_1.0.5.bb`

```
SUMMARY = "Minimalistic user-space Netlink utility library"
DESCRIPTION = "Minimalistic user-space library oriented to Netlink developers, providing
    functions for common tasks in parsing, validating, and constructing both the Netlink header and TLVs."
HOMEPAGE = "https://www.netfilter.org/projects/libmnl/index.html"
SECTION = "libs"
LICENSE = "LGPL-2.1-or-later"
LIC_FILES_CHKSUM = "file://COPYING;md5=4fbd65380cdd255951079008b364516c"

SRC_URI = "https://netfilter.org/projects/libmnl/files/libmnl-${PV}.tar.bz2"
SRC_URI[sha256sum] = "274b9b919ef3152bfb3da3a13c950dd60d6e2bcd54230ffeca298d03b40d0525"

inherit autotools pkgconfig

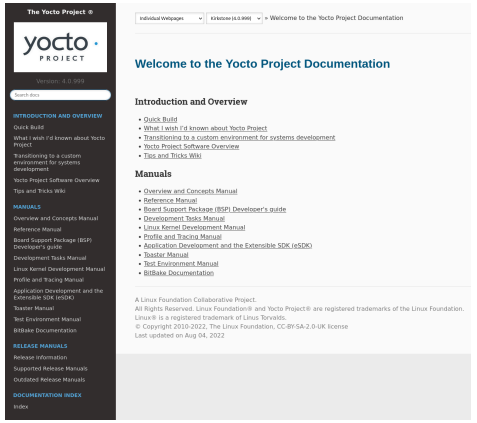
BBCLASSEXTEND = "native"
```

- Recipe to build libmnl
- Build system based on *autotools* → `inherit autotools`
- Available both for the target and the host → `BBCLASSEXTEND = "native"`



Yocto resources

- Official website: https://www.yoctoproject.org/
- Release information: https://wiki.yoctoproject.org/wiki/Releases
- Official documentation: https://docs.yoctoproject.org/
  - Maintained by Bootlin engineers!
- Complete *Yocto Project and OpenEmbedded system development* training course from Bootlin
  - https://bootlin.com/training/yocto/
  - Freely available training materials







Buildroot vs. Yocto: a few key differences

- What it builds
  - **Yocto**: builds a distribution, with binary packages and a package management system
  - **Buildroot**: builds a fixed functionality root filesystem, no binary packages
  - Note: binary packages are not necessarily a good thing for embedded!



Buildroot vs. Yocto: a few key differences

- What it builds
- Configuration
  - **Yocto**: flexible, powerful but complex configuration description
  - **Buildroot**: very simple configuration system, but sometimes limited



Buildroot vs. Yocto: a few key differences

- What it builds
- Configuration
- Build strategy
  - **Yocto**: complex and heavy logic, but with efficient caching of artifacts and "rebuild only what's needed" features
  - **Buildroot**: simple but somewhat dumb logic, no caching of built artifacts, full rebuilds needed for some config changes



Buildroot vs. Yocto: a few key differences

- What it builds
- Configuration
- Build strategy
- Ecosystem
  - **Yocto**: (relatively) small common base in OpenEmbedded, lots of features supported in third party layers →lots of things, but varying quality
  - **Buildroot**: everything in one tree →perhaps less things, but more consistent quality



Buildroot vs. Yocto: a few key differences

- What it builds
- Configuration
- Build strategy
- Ecosystem
- Complexity/learning curve
  - **Yocto**: admittedly steep learning curve, *bitbake* remains a magic black box for most people
  - **Buildroot**: much smoother and shorter learning curve, the tool is simple to approach, and reasonably simple to understand



Buildroot vs. Yocto: a few key differences

- What it builds
- Configuration
- Build strategy
- Ecosystem
- Complexity/learning curve
- And also a matter of personal taste/preference, as often when choosing tools



OpenWrt

- Another Embedded Linux build system
- Derived from Buildroot a *very* long time ago
  - Now completely different, except for the use of *Kconfig* and *make*
- Targeted at building firmware for WiFi routers and other networking equipment
- Unlike Buildroot or Yocto that leave a lot of flexibility to the user in defining the system architecture, OpenWrt makes a lot of set in stone decisions:
  - *musl* is the C library
  - an OpenWrt specific init system
  - an OpenWrt specific inter-process communication bus
  - a Web UI specific to OpenWrt
- The aim of OpenWrt is to build a final product out of the box, with support for popular networking products and development boards
- `https://openwrt.org/`



Working with distributions



Binary distributions

- Many popular Linux desktop/server distributions have support for embedded architectures
  - Debian: ARMv5, ARMv7, ARM64, i386, x86-64, MIPS, PowerPC, RISC-V in progress
  - Ubuntu: ARMv7, ARM64, x86-64, RISC-V (initial support), PowerPC64 little-endian
  - Fedora: ARMv7, ARM64, x86-64, MIPS little-endian, PowerPC64 little-endian, RISC-V
- Some more specialized Linux distributions as well
  - Raspberry Pi OS, a Debian derivative targeted at RaspberryPi platforms
  - Alpine Linux, a lightweight distribution, based on *musl* and *BusyBox*, ARMv7, ARM64, i386, x86-64, PowerPC64 little-endian



Binary distributions pitfalls

- Be careful when using a binary distribution on how you create your system image, and how reproducible this process is
- We have seen projects use the following (bad) procedure:
  - Install a binary distribution manually on their target hardware
  - Install all necessary packages by hand
  - Compile the final applications on the target
  - Tweak configuration files directly on the target
  - Then duplicate the resulting SD card for all other boards
- This process is really bad as:
  - it is not reproducible
  - it requires installing many more things on the target than needed (development tools), increasing the footprint, the attack surface and the maintenance effort
- If you end up using a binary distribution in production, make sure you have an automated and reproducible process to generate the complete image, ready to flash on your target.



Debian/Ubuntu image building tools

ELBE

- **E.**mbedded **L.**inux **B.**uild **E.**nvironment
- Implemented in Python
- Uses an XML file as input to describe the system to generate
- Can use pre-built packages from Debian/Ubuntu repositories, but can also cross-compile and install additional packages
- https://elbe-rfs.org/
- Building Embedded Debian and Ubuntu Systems with ELBE talk
- ELBE: automated building of Ubuntu images for a Raspberry Pi 3B

DebOS

- Debian OS images builder
- Implemented in Go
- Uses a YAML file as input to describe the system to generate
- Creating Debian-Based Embedded Systems in the Cloud Using Debos talk
- https://github.com/go-debos/debos



Android

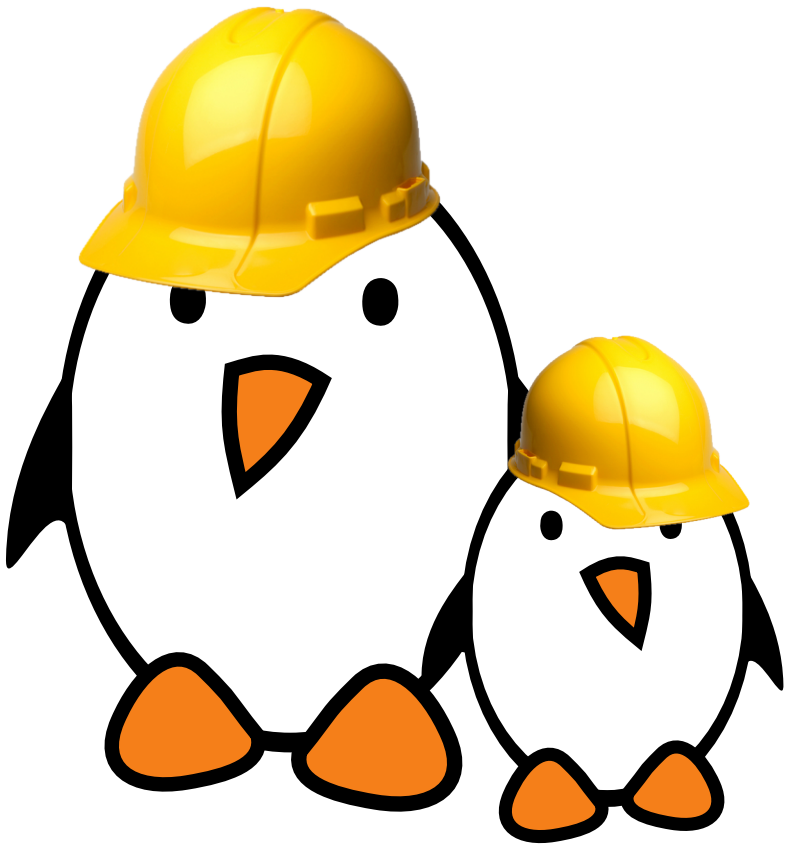
- The obviously highly popular mobile operating system
- Uses the Linux kernel
- Most of the user-space is completely different from a normal embedded Linux system
  - Most components rewritten by Google
  - *bionic* C library
  - Custom *init* system and device management
  - Custom IPC mechanism, custom display stack, custom multimedia stack
  - Custom build system
- Android pitfalls for industrial embedded systems
  - Large footprint, and resource hungry
  - Complexity and build time
  - Maintenance issues: difficult to upgrade to newer releases due to increasing hardware requirements
- Embedded Android Training course from Opersys, with freely available training materials



Automotive Grade Linux, Tizen

- Industry groups collaborate around the creation of embedded Linux distributions targeting specific markets
  - These are regular embedded Linux systems, usually based on Yocto, with a selection of relevant open-source software components
  - Fund the development of missing features in existing components, or development of new software components
- Automotive Grade Linux
  - Linux Foundation project
  - *Collaborative open source project that is bringing together automakers, suppliers and technology companies to accelerate the development and adoption of a fully open software stack for the connected car*
  - https://www.automotivelinux.org/
- Tizen
  - Linux Foundation project too
  - Operating system targeting TVs, wearables, phones, in-vehicle infotainment, based on HTML5 applications.
  - https://www.tizen.org/

Practical lab - System build with Buildroot



Time to start the practical lab!

- Using Buildroot to rebuild the same basic system plus a sound playing server (*MPD*) and a client to control it (*mpc*).
- Overlaying the root filesystem built by Buildroot
- Driving music playback, directly from the target, and then remotely through an MPD client on the host machine.
- Analyzing dependencies between packages.
- Building *evtest* and using it to test the Nunchuk device driver.